



Docker 핵심 프로세스 및 명령어 총정리

1. Docker 전체 처리 흐름 요약

Docker는 애플리케이션과 실행 환경을 컨테이너라는 격리된 공간에 묶어 실행하는 도구다. 전체적인 처리 흐름은 환경 정의, 이미지 빌드, 컨테이너 실행 및 관리의 순서로 진행된다.



- **환경 정의 (Dockerfile)**: 베이스 이미지, 설치할 패키지, 소스코드 복사 위치, 컨테이너 시작 명령 등을 설계도 파일에 작성한다.
- **이미지 빌드 (docker build)**: 작성한 설계도를 바탕으로 독립적으로 실행 가능한 파일 스냅샷인 이미지를 생성한다.
- **컨테이너 실행 (docker run / docker-compose up)**: 빌드된 이미지를 격리된 프로세스로 구동하여 실제 컨테이너 인스턴스를 만든다.
- **운영 및 디버깅 (docker exec / docker logs)**: 실행 중인 컨테이너 내부에 접속해 명령을 내리거나 로그를 보며 상태를 관리하고, 종료 후 폐기한다.

2. 주요 과정 및 핵심 코드 정리

① 이미지 빌드 및 최적화 과정

나만의 실행 환경을 직접 구축하고 스냅샷 이미지로 만들어내는 단계다.

```
# Dockerfile 기본 구조 예시
FROM ros:humble                                # 1. 기반 이미지 지정
RUN apt-get update && apt-get install -y python3-pip # 2. 빌드 중 패키지 설치 (레이어로 저장)
WORKDIR /workspace                             # 3. 기본 작업 폴더 설정
COPY . .                                         # 4. 호스트 파일 복사 (자주 바뀌므로 뒤에 배치)
CMD ["bash"]                                    # 5. 컨테이너 시작 시 실행 명령
```

- **.dockerignore** 적용: 이미지 빌드 시 불필요한 파일이나 무거운 폴더(`build/` , `install/` , `log/` , `.git`)를 빌드 범위에서 제외하여 빌드 속도를 높이고 용량을 줄임.
- `docker build -t my-app .` : 현재 폴더(`.`)를 기준으로 `my-app` 이라는 이름의 이미지를 빌드함.

② 단일 컨테이너 제어 및 디버깅 과정

생성된 이미지를 실행하고 구동 중인 컨테이너의 상태를 점검하거나 내부에 접속하는 단계다.

- `docker run -it ubuntu bash` : 대화형 터미널(`-it`)을 열어 Ubuntu 컨테이너를 실행하고 bash 셸을 구동함.
- `docker run -d nginx` : 터미널을 점유하지 않고 백그라운드(`-d`)에서 컨테이너를 실행함.
- `docker exec -it my-container bash` : 이미 실행 중인 컨테이너 내부에 추가로 접속하여 터미널 명령을 수행함.
- `docker ps` : 현재 실행 중인 컨테이너 목록 확인.
- `docker ps -a` : 종료되거나 중지된 컨테이너를 포함한 전체 목록 확인.
- `docker logs -f my-container` : 컨테이너 내부 로그를 실시간(`-f`)으로 계속 추적함.
- `docker stats` : 컨테이너의 CPU, 메모리 리소스 사용량을 실시간 확인.
- `docker inspect my-container` : 컨테이너의 IP 주소, 마운트 정보 등 상세 설정을 JSON 형태로 조회.
- `docker stop my-container` : 실행 중인 컨테이너 중지.
- `docker rm my-container` : 중지된 컨테이너 프로세스 삭제.
- `docker rmi my-image` : 로컬에 저장된 Docker 이미지 삭제.

③ 원격 배포 및 저장소 연동 과정

로컬에서 작업한 이미지를 공유하거나 외부 저장소에서 환경을 가져오는 단계다.

- `docker pull ros:humble` : Docker Hub 원격 저장소에서 이미지를 로컬로 내려받음.
- `docker push my-registry/my-app` : 로컬에서 생성한 이미지를 원격 저장소에 업로드함.
- `docker system prune -a` : 사용하지 않는 컨테이너, 네트워크, 이미지 등을 대량 정리하여 디스크 용량을 확보함 (주의 필요).

④ 데이터 보존 및 네트워크 설정 과정

격리된 컨테이너의 한계를 넘어 데이터를 영구 저장하고 외부 네트워크와 연결하는 단계다.

- **네트워크 포트 매핑 (`-p 8080:80`)**: 왼쪽이 내 PC(호스트) 포트, 오른쪽이 컨테이너 내부 포트임. 외부에서 브라우저 등으로 컨테이너 내부 서비스에 접근할 때 필수임.
- **볼륨 (`-v 볼륨명:컨테이너경로`)**: Docker가 내부적으로 안전하게 관리하는 보관함 저장소. 컨테이너 생명주기와 분리되어 데이터가 영구 보존됨.
- **바인드 마운트 (`-v 호스트경로:컨테이너경로`)**: 내 PC의 특정 폴더를 컨테이너와 직접 연결함. 호스트에서 소스코드를 수정하면 컨테이너 내부 반영이 즉시 이루어지므로 실습 및 개발 시 유용함.

⑤ docker-compose 다중 컨테이너 관리 과정

여러 개의 컨테이너 서비스와 이들이 사용하는 네트워크, 볼륨 설정을 `docker-compose.yaml` 파일 하나로 일괄 제어하는 단계다.

```
# docker-compose.yaml 구조 예시
services:
  ros2_app:
    image: ros:humble
    stdin_open: true           # 터미널 대화형 사용 설정
    tty: true                  # tty 개방
    ports:
      - "8080:80"             # 포트 매핑 (따옴표 처리가 안전)
    volumes:
      - ./ros2_ws:/workspace   # 바인드 마운트 연동
    environment:
      ROS_DOMAIN_ID: "30"      # 내부 환경변수 주입
    command: sleep infinity    # 컨테이너 자동 종료 방지용 대기 명령
```

- `docker-compose up -d` : 작성된 모든 서비스를 백그라운드에서 한 번에 생성 및 실행.
- `docker-compose ps` : 해당 Compose 프로젝트 기준으로 구동 중인 서비스 상태 확인.
- `docker-compose exec ros2_app bash` : 실행 중인 특정 서비스 내부로 진입.
- `docker-compose logs -f` : Compose 전체 서비스의 로그를 실시간 모니터링.
- `docker-compose down` : 생성된 컨테이너와 가상 네트워크를 일괄 중지하고 삭제 처리.
- `docker-compose down -v` : 컨테이너 정리와 동시에 사용된 named 볼륨 데이터까지 영구 삭제하므로 데이터 소실에 극도로 주의해야 함.

3. 실전 트러블슈팅 및 핵심 디버깅 프로세스

ROS2 실습 환경 등에서 에러가 발생했을 때 따르는 표준 점검 순서다.

1. `docker ps -a` 로 컨테이너가 Exited 상태로 종료되었는지 상태를 먼저 확인한다.
2. 컨테이너가 종료되었다면 `docker logs <컨테이너명>` 으로 메인 프로세스의 종료 원인 및 에러 로그를 분석한다.
3. 컨테이너가 정상 구동 중이라면 `docker exec -it <컨테이너명> bash` 로 내부에 접속한다.
4. 내부에서 ROS2 명령어가 실행되지 않는다면 `source /opt/ros/humble/setup.bash` 를 실행하여 환경 설정을 로드한다.
5. `printenv | grep ROS` 명령어를 통해 현재 설정된 환경변수와 `ROS_DOMAIN_ID` 를 확인한다.
6. 통신 혼선이나 노드 미발견 문제가 있다면 `ROS_DOMAIN_ID` 가 다른 수강생과 매칭되어 cross-talk가 발생했는지 비교 점검한다.